

5. （可选）优化本文档所展示的量化、剪枝、知识蒸馏相关代码，以实现更好的性能

实验环境

- 硬件：电脑（至少 8GB 内存），Jetson Orin NX 设备，网线，路由器
- 软件：XShell 或者 Termius 等终端工具，Python 3.7+，PyTorch 1.8+，JupyterNotebook，依赖如下

PowerShell

安装依赖

在自己的电脑上安装以下 python 库

```
pip install torch torchvision numpy matplotlib tqdm
```

实验步骤

1. 基础模型训练

首先，新建一个 JupyterNotebook 文件用以运行，随后，运行以下代码训练一个基准 ResNet-18 模型用于后续轻量化比较。

此步骤演示代码 epoch 设定为 5，若算力充足，可考虑将 epoch 调大以提高模型精度。

```
python

import torch

import torch.nn as nn

import torch.optim as optim

import torchvision

import torchvision.transforms as transforms

from torchvision.models import resnet18

import time

import numpy as np

import matplotlib.pyplot as plt
```

```
from tqdm import tqdm

# 设置随机种子确保可复现性
torch.manual_seed(42)

# 数据预处理
transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])

# 加载 CIFAR-10 数据集
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
download=True, transform=transform_train)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128,
shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform_test)
testloader = torch.utils.data.DataLoader(testset, batch_size=100,
shuffle=False, num_workers=2)
```

```
# 加载 ResNet-18 模型并修改最后一层以适应 CIFAR-10
model = resnet18(pretrained=False)

model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
bias=False)

model.maxpool = nn.Identity() # 移除 maxpool 层, 因为 CIFAR-10 图像较小
model.fc = nn.Linear(model.fc.in_features, 10) # CIFAR-10 有 10 个类别


device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = model.to(device)


# 定义损失函数和优化器
criterion = nn.CrossEntropyLoss()

optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
weight_decay=5e-4)

scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=200)


# 训练函数
def train(epoch):
    model.train()

    train_loss = 0

    correct = 0

    total = 0


    with tqdm(trainloader, unit="batch") as tepoch:
        for inputs, targets in tepoch:
            tepoch.set_description(f"Epoch {epoch}")
```

```

        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()

        outputs = model(inputs)

        loss = criterion(outputs, targets)

        loss.backward()

        optimizer.step()

    train_loss += loss.item()

    _, predicted = outputs.max(1)

    total += targets.size(0)

    correct += predicted.eq(targets).sum().item()

    tepoch.set_postfix(loss=train_loss/(tePOCH.n+1),
accuracy=100.*correct/total)

    return train_loss/len(trainloader), 100.*correct/total

```

测试函数

```

def test(epoch):
    model.eval()

    test_loss = 0

    correct = 0

    total = 0

    with torch.no_grad():
        for inputs, targets in testloader:

            inputs, targets = inputs.to(device), targets.to(device)

            outputs = model(inputs)

```

```

        loss = criterion(outputs, targets)

        test_loss += loss.item()
        _, predicted = outputs.max(1)
        total += targets.size(0)
        correct += predicted.eq(targets).sum().item()

    print(f'测试结果 - Epoch: {epoch}, Loss: {test_loss/len(testloader):.3f},
    Accuracy: {100.*correct/total:.3f}%')

    return test_loss/len(testloader), 100.*correct/total

# 训练模型
epochs = 5 # 实际训练应增加到 30-50 轮，这里为了实验效率设置较小
train_losses, test_losses = [], []
train_accs, test_accs = [], []

start_time = time.time()
for epoch in range(1, epochs + 1):
    train_loss, train_acc = train(epoch)
    test_loss, test_acc = test(epoch)
    scheduler.step()

    train_losses.append(train_loss)
    test_losses.append(test_loss)
    train_accs.append(train_acc)
    test_accs.append(test_acc)

```

```
training_time = time.time() - start_time

print(f'基准模型训练完成，总耗时: {training_time:.2f}秒')

# 保存基准模型

torch.save(model.state_dict(), 'resnet18_baseline.pth')
```

```
Epoch 1: 100%|██████████| 391/391 [10:16<00:00, 1.58s/batch, accuracy=25.5, loss=2.1]
测试结果 - Epoch: 1, Loss: 1.614, Accuracy: 39.780%
Epoch 2: 100%|██████████| 391/391 [13:54<00:00, 2.13s/batch, accuracy=44.8, loss=1.5]
测试结果 - Epoch: 2, Loss: 1.375, Accuracy: 50.470%
Epoch 3: 100%|██████████| 391/391 [13:35<00:00, 2.08s/batch, accuracy=55.8, loss=1.22]
测试结果 - Epoch: 3, Loss: 1.072, Accuracy: 61.310%
Epoch 4: 100%|██████████| 391/391 [13:41<00:00, 2.10s/batch, accuracy=64.4, loss=0.999]
测试结果 - Epoch: 4, Loss: 1.023, Accuracy: 65.180%
Epoch 5: 100%|██████████| 391/391 [13:40<00:00, 2.10s/batch, accuracy=70.8, loss=0.827]
测试结果 - Epoch: 5, Loss: 1.168, Accuracy: 62.540%
基准模型训练完成，总耗时: 4191.63秒
```

2. 模型量化

接下来，我们对模型进行量化，将 FP32 参数转换为 INT8

```
python

import torch

import torch.nn as nn

import torchvision

import torch.quantization

import os

import time

import numpy as np

from torchvision import transforms

from torchvision.models import resnet18

# Define transforms for the data

transform = transforms.Compose([

    transforms.ToTensor(),
```

```

        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])

# Load CIFAR-10 dataset
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=128,
                                          shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                       download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=128,
                                         shuffle=False, num_workers=2)

# Function to measure inference time
def measure_inference_time(model, device, num_runs=100):
    dummy_input = torch.randn(1, 3, 32, 32).to(device)

    # Warm-up
    for _ in range(10):
        _ = model(dummy_input)

    # Measure
    start_time = time.time()
    for _ in range(num_runs):
        _ = model(dummy_input)
    end_time = time.time()

```

```
avg_time = (end_time - start_time) / num_runs * 1000 # ms
return avg_time

# 准备用于量化的模型
model_fp32 = resnet18(pretrained=False)
model_fp32.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
bias=False)
model_fp32.maxpool = nn.Identity()
model_fp32.fc = nn.Linear(model_fp32.fc.in_features, 10)

# 加载训练好的权重
model_fp32.load_state_dict(torch.load('resnet18_baseline.pth'))

# 设置模型为评估模式
model_fp32.eval()

# 测试量化模型性能
device_cpu = torch.device('cpu')

def test_model(model, device):
    model.eval()
    test_loss = 0
    correct = 0
    total = 0

    criterion = nn.CrossEntropyLoss()

    with torch.no_grad():
```

```

for inputs, targets in testloader:

    inputs, targets = inputs.to(device), targets.to(device)

    outputs = model(inputs)

    loss = criterion(outputs, targets)

    test_loss += loss.item()

    _, predicted = outputs.max(1)

    total += targets.size(0)

    correct += predicted.eq(targets).sum().item()

print(f'模型测试结果 - Loss: {test_loss/len(testloader):.3f}, Accuracy:
{100.*correct/total:.3f}%')

return test_loss/len(testloader), 100.*correct/total

# 进行动态量化 - 比静态量化兼容性更好
quantized_model = torch.quantization.quantize_dynamic(
    model_fp32, # the original model
    {nn.Linear, nn.Conv2d}, # a set of layers to dynamically quantize
    dtype=torch.qint8 # the target dtype for quantized weights
)

# 保存量化模型
torch.save(quantized_model.state_dict(), 'resnet18_quantized_dynamic.pth')

# 测试量化模型
quantized_loss, quantized_acc = test_model(quantized_model, device_cpu)

```

```
# 测量量化模型大小

quantized_size = os.path.getsize('resnet18_quantized_dynamic.pth') / (1024 *
1024) # MB

print(f'量化模型大小: {quantized_size:.2f} MB')


# 测量量化模型推理时间

quantized_inference_time = measure_inference_time(quantized_model,
device_cpu)

print(f'量化模型平均推理时间: {quantized_inference_time:.3f} ms')


# 对比原始 FP32 模型性能

fp32_inference_time = measure_inference_time(model_fp32, device_cpu)

print(f'FP32 模型平均推理时间: {fp32_inference_time:.3f} ms')

print(f'速度提升: {fp32_inference_time / quantized_inference_time:.2f}x')


# 保存为 TorchScript 模型

try:

    scripted_quantized_model = torch.jit.script(quantized_model)

    torch.jit.save(scripted_quantized_model,
'resnet18_quantized_dynamic_scripted.pt')

    print("成功保存 TorchScript 模型")

except Exception as e:

    print(f"TorchScript 保存失败: {e}")
```

```
Files already downloaded and verified
Files already downloaded and verified
模型测试结果 - Loss: 2.277, Accuracy: 39.030%
量化模型大小: 42.69 MB
量化模型平均推理时间: 12.174 ms
FP32模型平均推理时间: 11.707 ms
速度提升: 0.96x
成功保存TorchScript模型
```

3. 模型剪枝

现在我们对模型进行剪枝，移除不重要的权重

```
python

import torch.nn.utils.prune as prune

# 加载基准模型
pruned_model = resnet18(pretrained=False)
pruned_model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,
bias=False)
pruned_model.maxpool = nn.Identity()
pruned_model.fc = nn.Linear(pruned_model.fc.in_features, 10)
pruned_model.load_state_dict(torch.load('resnet18_baseline.pth'))
pruned_model = pruned_model.to(device)

# 对模型进行全局剪枝（例如剪掉 30%的权重）
pruning_amount = 0.3

# 对所有卷积层和全连接层进行剪枝
for module in pruned_model.modules():
    if isinstance(module, torch.nn.Conv2d) or isinstance(module,
```

```

torch.nn.Linear):

    prune.l1_unstructured(module, name='weight', amount=pruning_amount)

    # 使剪枝永久化
    prune.remove(module, 'weight')

# 微调剪枝后的模型
def finetune(model, epochs=3):
    model.train()

    optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9,
weight_decay=5e-4)

    for epoch in range(1, epochs + 1):
        with tqdm(trainloader, unit="batch") as tepoch:
            for inputs, targets in tepoch:
                tepoch.set_description(f"微调 Epoch {epoch}")

                inputs, targets = inputs.to(device), targets.to(device)
                optimizer.zero_grad()
                outputs = model(inputs)
                loss = criterion(outputs, targets)
                loss.backward()
                optimizer.step()

            # 计算并显示准确率
            _, predicted = outputs.max(1)
            accuracy = 100. * predicted.eq(targets).sum().item() /
targets.size(0)

            tepoch.set_postfix(loss=loss.item(), accuracy=accuracy)

```

```

# 微调剪枝后的模型
finetune(pruned_model)

# 保存剪枝模型
torch.save(pruned_model.state_dict(), 'resnet18_pruned.pth')

# 测试剪枝模型性能
pruned_loss, pruned_acc = test(0) # 这里的 epoch 参数不重要

# 测量剪枝模型大小
pruned_size = os.path.getsize('resnet18_pruned.pth') / (1024 * 1024) # MB
print(f'剪枝模型大小: {pruned_size:.2f} MB')

# 测量剪枝模型推理时间
pruned_inference_time = measure_inference_time(pruned_model, device)
print(f'剪枝模型平均推理时间: {pruned_inference_time:.3f} ms')

```

```

微调 Epoch 1: 100%|██████████| 391/391 [09:49<00:00, 1.51s/batch, accuracy=87.5, loss=0.485]
微调 Epoch 2: 100%|██████████| 391/391 [09:56<00:00, 1.53s/batch, accuracy=85, loss=0.366]
微调 Epoch 3: 100%|██████████| 391/391 [09:47<00:00, 1.50s/batch, accuracy=85, loss=0.461]
测试结果 - Epoch: 0, Loss: 2.274, Accuracy: 39.080%
剪枝模型大小: 42.70 MB
剪枝模型平均推理时间: 12.579 ms

```

4. 知识蒸馏

最后，我们使用知识蒸馏训练一个小型模型

```

python

# 定义教师模型（使用之前训练好的 ResNet-18）和学生模型（使用更小的网络）
class SmallModel(nn.Module):

    def __init__(self, num_classes=10):

```

```

super(SmallModel, self).__init__()

# 一个简单的 CNN 网络

self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
self.pool = nn.MaxPool2d(2, 2)
self.fc1 = nn.Linear(32 * 8 * 8, 128)
self.fc2 = nn.Linear(128, num_classes)
self.relu = nn.ReLU()

def forward(self, x):
    x = self.pool(self.relu(self.conv1(x)))
    x = self.pool(self.relu(self.conv2(x)))
    x = x.view(-1, 32 * 8 * 8)
    x = self.relu(self.fc1(x))
    x = self.fc2(x)
    return x

# 创建教师模型和学生模型
teacher_model = model # 使用之前训练的 ResNet-18
student_model = SmallModel().to(device)

# 定义蒸馏损失函数
def distillation_loss(student_logits, teacher_logits, labels, T=3.0, alpha=0.8):
    """
    计算蒸馏损失
    T: 温度参数
    alpha: 混合参数，控制蒸馏损失和原始交叉熵损失的权重
    """

```

```

# 软目标损失
distillation_loss = nn.KLDivLoss(reduction='batchmean')(
    nn.functional.log_softmax(student_logits / T, dim=1),
    nn.functional.softmax(teacher_logits / T, dim=1)
) * (T * T)

# 硬目标损失
student_loss = criterion(student_logits, labels)

# 混合损失
loss = alpha * distillation_loss + (1 - alpha) * student_loss
return loss

# 蒸馏训练函数
def train_distillation(student_model, teacher_model, epochs=10):
    teacher_model.eval() # 教师模型处于评估模式
    student_model.train()

    optimizer = optim.SGD(student_model.parameters(), lr=0.01,
momentum=0.9, weight_decay=5e-4)

    for epoch in range(1, epochs + 1):
        with tqdm(trainloader, unit="batch") as tepoch:
            for inputs, targets in tepoch:
                tepoch.set_description(f"蒸馏 Epoch {epoch}")

                inputs, targets = inputs.to(device), targets.to(device)

```

```
# 获取教师模型输出

with torch.no_grad():
    teacher_outputs = teacher_model(inputs)

# 获取学生模型输出

student_outputs = student_model(inputs)

# 计算蒸馏损失

loss = distillation_loss(student_outputs, teacher_outputs, targets)

# 反向传播

optimizer.zero_grad()
loss.backward()
optimizer.step()

# 计算并显示准确率

_, predicted = student_outputs.max(1)
accuracy = 100. * predicted.eq(targets).sum().item() /
targets.size(0)

teepoch.set_postfix(loss=loss.item(), accuracy=accuracy)

return student_model

# 训练学生模型

distilled_model = train_distillation(student_model, teacher_model)

# 保存蒸馏模型

torch.save(distilled_model.state_dict(), 'small_distilled.pth')
```

定义测试函数

```
def test_model(model, name):  
    model.eval()  
    correct = 0  
    total = 0  
  
    with torch.no_grad():  
        for inputs, targets in testloader:  
            inputs, targets = inputs.to(device), targets.to(device)  
            outputs = model(inputs)  
            _, predicted = outputs.max(1)  
            total += targets.size(0)  
            correct += predicted.eq(targets).sum().item()  
  
    accuracy = 100. * correct / total  
    print(f'{name} 模型测试准确率: {accuracy:.2f}%')  
    return accuracy
```

测试蒸馏模型

```
distilled_acc = test_model(distilled_model, "蒸馏")
```

测量蒸馏模型大小

```
distilled_size = os.path.getsize('small_distilled.pth') / (1024 * 1024) # MB  
print(f'蒸馏模型大小: {distilled_size:.2f} MB')
```

测量蒸馏模型推理时间

```
distilled_inference_time = measure_inference_time(distilled_model, device)
```

```
print(f'蒸馏模型平均推理时间: {distilled_inference_time:.3f} ms')
```

```
蒸馏 Epoch 1: 100%|██████████| 391/391 [03:15<00:00, 2.00batch/s, accuracy=25, loss=1.22]
蒸馏 Epoch 2: 100%|██████████| 391/391 [03:22<00:00, 1.93batch/s, accuracy=30, loss=0.957]
蒸馏 Epoch 3: 100%|██████████| 391/391 [03:22<00:00, 1.93batch/s, accuracy=48.8, loss=0.764]
蒸馏 Epoch 4: 100%|██████████| 391/391 [03:20<00:00, 1.95batch/s, accuracy=51.2, loss=0.724]
蒸馏 Epoch 5: 100%|██████████| 391/391 [03:21<00:00, 1.94batch/s, accuracy=38.8, loss=0.755]
蒸馏 Epoch 6: 100%|██████████| 391/391 [03:23<00:00, 1.92batch/s, accuracy=33.8, loss=0.733]
蒸馏 Epoch 7: 100%|██████████| 391/391 [03:20<00:00, 1.95batch/s, accuracy=45, loss=0.642]
蒸馏 Epoch 8: 100%|██████████| 391/391 [03:18<00:00, 1.97batch/s, accuracy=35, loss=0.791]
蒸馏 Epoch 9: 100%|██████████| 391/391 [03:23<00:00, 1.92batch/s, accuracy=45, loss=0.602]
蒸馏 Epoch 10: 100%|██████████| 391/391 [03:27<00:00, 1.89batch/s, accuracy=51.2, loss=0.622]
蒸馏 模型测试准确率: 41.01%
蒸馏模型大小: 1.03 MB
蒸馏模型平均推理时间: 0.583 ms
```

5. 性能对比与可视化

```
python

import matplotlib.pyplot as plt

# 配置 matplotlib 使用支持中文的字体
plt.rcParams['font.sans-serif'] = ['SimHei'] # 使用 SimHei 字体
plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题

# 比较不同模型的性能
models = ['基准模型', '量化模型', '剪枝模型', '蒸馏模型']
accuracies = [test_accs[-1], quantized_acc, pruned_acc, distilled_acc]
model_sizes = [baseline_size, quantized_size, pruned_size, distilled_size]
inference_times = [baseline_inference_time, quantized_inference_time,
pruned_inference_time, distilled_inference_time]

# 创建性能对比表格
print("\n 模型性能对比:")

print("-" * 80)

print(f"{'模型':<10} | {'准确率 (%)':<12} | {'大小 (MB)':<12} | {'推理时间 (ms)':<15} |
```

```
{ '大小压缩比':<12} | { '速度提升':<10} ")

print("-" * 80)

for i, model_name in enumerate(models):

    size_ratio = model_sizes[i] / baseline_size

    speed_up = baseline_inference_time / inference_times[i]

    print(f"{'model_name':<10} | {'accuracies[i]:<12.2f'} | {'model_sizes[i]:<12.2f'} |
{'inference_times[i]:<15.3f'} | {'size_ratio:<12.2f'} | {'speed_up:<10.2f'}")

# 可视化性能对比
plt.figure(figsize=(15, 10))

# 准确率对比
plt.subplot(2, 2, 1)

plt.bar(models, accuracies, color=['blue', 'orange', 'green', 'red'])

plt.title('模型准确率对比')

plt.xlabel('模型')

plt.ylabel('准确率 (%)')

plt.ylim([min(accuracies) * 0.95, max(accuracies) * 1.02])

# 模型大小对比
plt.subplot(2, 2, 2)

plt.bar(models, model_sizes, color=['blue', 'orange', 'green', 'red'])

plt.title('模型大小对比')

plt.xlabel('模型')

plt.ylabel('大小 (MB)')

# 推理时间对比
```

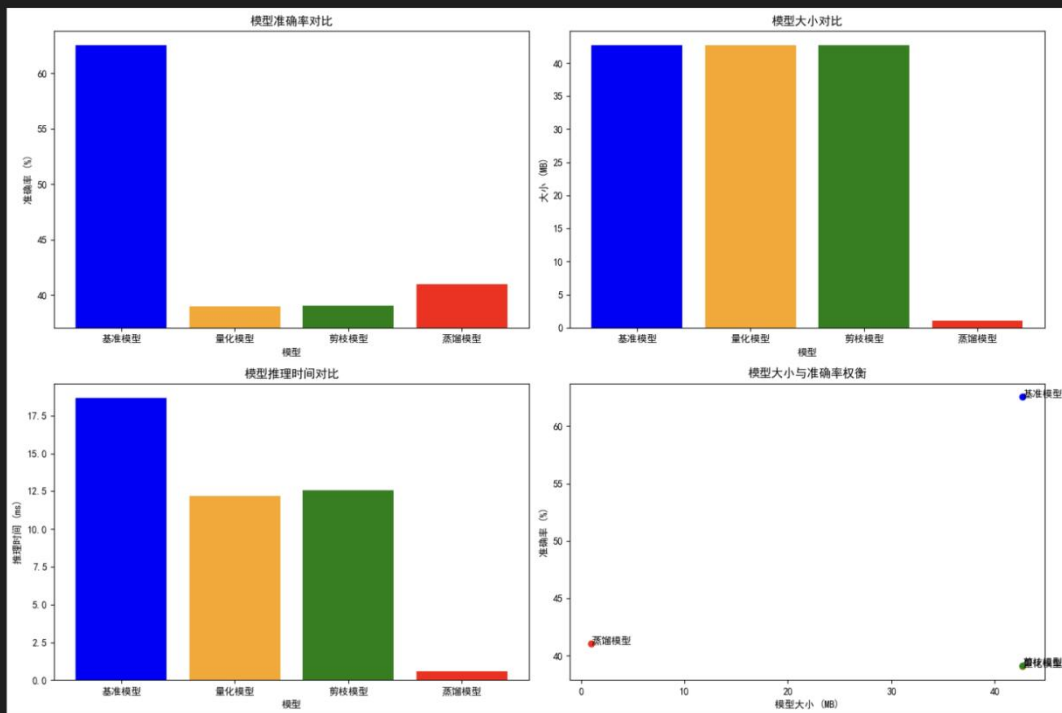
```
plt.subplot(2, 2, 3)
plt.bar(models, inference_times, color=['blue', 'orange', 'green', 'red'])
plt.title('模型推理时间对比')
plt.xlabel('模型')
plt.ylabel('推理时间 (ms)')

# 大小-准确率权衡
plt.subplot(2, 2, 4)
plt.scatter(model_sizes, accuracies, c=['blue', 'orange', 'green', 'red'])
for i, model_name in enumerate(models):
    plt.annotate(model_name, (model_sizes[i], accuracies[i]))
plt.title('模型大小与准确率权衡')
plt.xlabel('模型大小 (MB)')
plt.ylabel('准确率 (%)')

plt.tight_layout()
plt.savefig('model_comparison.png')
plt.show()
```

模型性能对比:

模型	准确率 (%)	大小 (MB)	推理时间 (ms)	大小压缩比	速度提升
基准模型	62.54	42.70	18.696	1.00	1.00
量化模型	39.03	42.69	12.174	1.00	1.54
剪枝模型	39.08	42.70	12.579	1.00	1.49
蒸馏模型	41.01	1.03	0.583	0.02	32.05



6. 模型导出

python

```
def export_to_onnx(model, model_path, onnx_path):
```

```
    """将 PyTorch 模型导出为 ONNX 格式"""
```

```
    model.load_state_dict(torch.load(model_path))
```

```
    model.eval()
```

```
    # 创建示例输入
```

```
    dummy_input = torch.randn(1, 3, 32, 32)
```

```
    # 导出 ONNX 模型
```

```
    torch.onnx.export(
```

```

    model,
    dummy_input,
    onnx_path,
    export_params=True,
    opset_version=11,
    do_constant_folding=True,
    input_names=['input'],
    output_names=['output'],
    dynamic_axes={'input': {0: 'batch_size'},
                  'output': {0: 'batch_size'}}
)

print(f"模型已导出至 {onnx_path}")

# 导出模型到 ONNX 格式
export_to_onnx(model, 'resnet18_baseline.pth', 'resnet18_baseline.onnx')
export_to_onnx(pruned_model, 'resnet18_pruned.pth', 'resnet18_pruned.onnx')
export_to_onnx(student_model, 'small_distilled.pth', 'small_distilled.onnx')

```

7. 在 NX 中运行模型性能测试

将以下代码保存到一个 py 文件中，如 `nx_inference_benchmark.py`

确保所有模型文件都已复制到同一目录下：

ONNX 模型: `resnet18_baseline.onnx`, `resnet18_pruned.onnx`, `small_distilled.onnx`

TorchScript 模型: `resnet18_quantized_dynamic_scripted.pt`

安装依赖 `pip install onnxruntime-gpu torch numpy matplotlib pillow`

运行基准测试 `python3 nx_inference_benchmark.py`

```
python
import os
```

```
import time
import numpy as np
import torch
import onnxruntime as ort
import matplotlib.pyplot as plt
from PIL import Image
import torchvision.transforms as transforms

# 配置 matplotlib 中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 设置图像预处理
preprocess = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

def test_onnx_model(model_path, num_runs=100):
    """测试 ONNX 模型在 NX 上的推理时间"""
    print(f"测试 ONNX 模型: {model_path}")

    # 检查文件是否存在
    if not os.path.exists(model_path):
        print(f"错误: 找不到模型文件 {model_path}")
        return None
```

```
# 创建 ONNX 运行时会话

try:
    # 为 Jetson 创建优化的 ONNX 会话
    sess_options = ort.SessionOptions()
    sess_options.graph_optimization_level =
ort.GraphOptimizationLevel.ORT_ENABLE_ALL

    # 使用 CUDA 执行提供程序以在 GPU 上运行
    providers = ['CUDAExecutionProvider', 'CPUExecutionProvider']

    session = ort.InferenceSession(model_path, sess_options,
providers=providers)


# 获取模型输入名称
input_name = session.get_inputs()[0].name


# 创建随机输入数据
dummy_input = np.random.randn(1, 3, 32, 32).astype(np.float32)


# 预热
print("预热中...")
for _ in range(10):
    _ = session.run(None, {input_name: dummy_input})


# 测量推理时间
print("测量中...")
inference_times = []
for _ in range(num_runs):
    start_time = time.time()
    _ = session.run(None, {input_name: dummy_input})
```

```

        inference_times.append((time.time() - start_time) * 1000) # ms

    avg_time = np.mean(inference_times)
    std_time = np.std(inference_times)

    print(f"平均推理时间: {avg_time:.2f} ms, 标准差: {std_time:.2f} ms")
    return avg_time

except Exception as e:
    print(f"运行 ONNX 模型时出错: {e}")
    return None

def test_torchscript_model(model_path, num_runs=100):
    """测试 TorchScript 模型在 NX 上的推理时间"""
    print(f"测试 TorchScript 模型: {model_path}")

    # 检查文件是否存在
    if not os.path.exists(model_path):
        print(f"错误: 找不到模型文件 {model_path}")
        return None

    try:
        # 加载 TorchScript 模型
        device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
        model = torch.jit.load(model_path, map_location=device)
        model.eval()

        # 创建随机输入数据

```

```

dummy_input = torch.randn(1, 3, 32, 32, device=device)

# 预热
print("预热中...")
with torch.no_grad():
    for _ in range(10):
        _ = model(dummy_input)

# 测量推理时间
print("测量中...")
inference_times = []
with torch.no_grad():
    for _ in range(num_runs):
        torch.cuda.synchronize() # 确保 GPU 操作完成
        start_time = time.time()
        _ = model(dummy_input)
        torch.cuda.synchronize() # 确保 GPU 操作完成
        inference_times.append((time.time() - start_time) * 1000) # ms

avg_time = np.mean(inference_times)
std_time = np.std(inference_times)

print(f"平均推理时间: {avg_time:.2f} ms, 标准差: {std_time:.2f} ms")
return avg_time

except Exception as e:
    print(f"运行 TorchScript 模型时出错: {e}")
    return None

```

```
def benchmark_all_models():
    """对所有模型进行基准测试并比较结果"""
    # 模型路径
    models = {
        '基准模型': 'resnet18_baseline.onnx',
        '剪枝模型': 'resnet18_pruned.onnx',
        '蒸馏模型': 'small_distilled.onnx',
        '量化模型': 'resnet18_quantized_dynamic_scripted.pt' # TorchScript 模型
    }

    # 存储结果
    results = {}

    # 运行测试
    for name, path in models.items():
        print(f"\n{'-'*30}")
        print(f"测试 {name}")
        print(f"{'-'*30}")

        if name == '量化模型':
            avg_time = test_torchscript_model(path)
        else:
            avg_time = test_onnx_model(path)

        if avg_time is not None:
            results[name] = avg_time
```

```
# 显示结果

print("\n" + "="*50)

print("Jetson NX 模型性能比较结果")

print("="*50)


if not results:

    print("没有可用的结果")

    return


# 计算加速比

baseline_time = results.get('基准模型')

if baseline_time:

    print(f"{'模型':<12} | {'推理时间 (ms)':<15} | {'相对基准模型加速比':<20}")

    print("-"*60)


    for name, time in results.items():

        speedup = baseline_time / time if time > 0 else float('nan')

        print(f"{'name':<12} | {'time':<15.2f} | {'speedup':<20.2f}x")

else:

    print(f"{'模型':<12} | {'推理时间 (ms)':<15}")

    print("-"*30)


    for name, time in results.items():

        print(f"{'name':<12} | {'time':<15.2f}")


# 绘制结果图表

model_names = list(results.keys())

times = list(results.values())
```

```
plt.figure(figsize=(12, 6))

# 推理时间柱状图
plt.subplot(1, 2, 1)
plt.bar(model_names, times, color=['blue', 'green', 'red', 'orange'])
plt.title('Jetson NX 模型推理时间')
plt.xlabel('模型')
plt.ylabel('推理时间 (ms)')
plt.xticks(rotation=45, ha='right')

# 加速比柱状图
if baseline_time:
    plt.subplot(1, 2, 2)
    speedups = [baseline_time / time if time > 0 else 0 for time in times]
    plt.bar(model_names, speedups, color=['blue', 'green', 'red', 'orange'])
    plt.title('相对基准模型加速比')
    plt.xlabel('模型')
    plt.ylabel('加速比')
    plt.xticks(rotation=45, ha='right')
    plt.axhline(y=1.0, color='r', linestyle='--')

plt.tight_layout()
plt.savefig('nx_model_comparison.png')
plt.show()

if __name__ == "__main__":
    benchmark_all_models()
```